Abstract

In recent years, the field of image generation has been revolutionized by the application of autoregressive transformers and DDPMs. These approaches model the process of image generation as a stepwise probabilistic process.

This paper describes a way to marry these two approaches so as to take advantage of both of their strengths. It applies this approach in the domain of speech synthesis to build TorToiSe - an expressive, multi-voice text-to-speech system. Finally, it shows that research in re-using pre-trained large language models for new tasks can also be applied to the speech domain with impressive results.

All model code and trained weights have been open-sourced at https://github.com/neonbjb/tortoise-tts.

Introduction

Text-to-speech

The field of text-to-speech (TTS) research has been largely constrained to the development of efficient models trained on relatively small datasets. This choice has been driven by:

- The desire to build efficient speech generation models that can be deployed at scale and thus must have a high sampling rate.
- The unavailability of very large, transcribed speech datasets.
- Challenges scaling the encoder-decoder model architectures traditionally used in TTS.

Notable works in the field include <>.

Neural Vocoders

Most modern text-to-speech systems operate on speech data that is encoded as a MEL spectrogram. There are many compelling reasons to operate in this encoding space, but for neural networks, the most compelling reason is that it is highly spatially compressed. The MEL configuration used by the Tacotron works, for example, operates at 256x compression.

Because of this fact, an entire body of research has been dedicated to finding high-quality ways to decode MEL spectrograms back into audio waveforms. A synthesizer that performs this task is generally called a “vocoder”.

Modern vocoders built on neural networks are incredibly sophisticated. They produce waveforms that are nearly indistinguishable from recorded waveforms to human ears, and they

are highly generalizable outside of their training set. I capitalize on this work by using the Univnet vocoder as a final stage for Tortoise.

Image generation

In the domain of image generation, much more focus has been applied to creating models that generate high-quality results, regardless of the sampling time. For the purposes of this paper, I would like to dive into two bodies of research:

DALL-E

The DALL-E framework showed how an autoregressive decoder can be applied to text-to-image generation. This is particularly appealing because of the vast quantity of research that has been poured into scaling decoder-only models in the NLP domain.

Two important problems persist with DALL-E: first, it partially relies on full-sequence self-attention, which carries a cost of O(N^2) compute and memory, where N is the sequence length. This is particularly troublesome when dealing with modalities like images or audio, which have large sequence lengths when dealt with naively.

Second, traditional autoregressive approaches requires operating in the discrete domain. Images are encoded into sequences of discrete tokens which DALL-E uses to build a powerful Bayesian model. This is an strength of DALL-E in terms of expressiveness, but comes at the cost of requiring a decoder which can convert these image tokens back into the pixel values that actually comprise an image. DALL-E 1 does this with a VQVAE decoder, which is prone to producing blurry images

DDPMs

The generative model space has long been plagued by models that either exhibit mean-seeking behavior (resulting in blurriness in the image domain, as an example) or mode-collapse (resulting in a lack of generalization).

Denoising diffusion probabilistic models (DDPMs) have recently arisen as the first type of generative model to exhibit neither of these undesirable traits. These models have been shown to be quite effective at using low-quality guidance signals to reconstruct the high-dimensional space that those guidance signals were derived from. Put another way, they are great at super-resolution.

There are two caveats to DDPMS:

1. Traditional approaches to DDPMs rely on the guidance signal being aligned with the output in some way. As a concrete example relevant to this paper, DDPMs cannot learn to convert text into audio signals because the text is unaligned with the audio.
2. DDPMs inherit their multi-modal inductive prior to an extended sampling process, where a neural network must be forward propagated many times to generate an output.

Research has been slowly chipping away at this limitation, but it seems likely that good multi-modal performance will always require the application of more compute.

Re-ranking

DALL-E introduced the process of “re-ranking” the outputs of autoregressive models. This process samples randomly from the autoregressive model picks the highest quality output of k outputs for downstream use.

Such a procedure requires a strong discriminator: a model that can tell good text<->image pairings from bad. As an example, DALL-E used CLIP, a model trained with a contrastive text and image pairing objective.

Methods

Joining Autoregressive Decoders and DDPMs

To review some of the conclusions drawn above:

- Autoregressive models are strong at converting between structured domains like vision, text and speech, while DDPMs struggle with alignment issues.
- DDPMs operate in the continuous domain which allows them to model expressive modalities, while autoregressive models are confined to discrete tokens.
- Both types of models have demonstrated the ability to scale performance with additional compute.

It becomes evident that when posed with a problem like generating continuous data like speech spectrograms or images, a marriage of these two approaches might have some distinct advantages.

Specifically, in inference, the autoregressive model will be used to convert a sequence of text tokens to a sequence of tokens representing the output space (in our case, speech tokens). The DDPM will then be used to decode these tokens into the a high quality representation of speech.

Applying Autoregression+DDPMs to TTS

To build out the previously proposed system, we need to train the following neural networks:

1. A VQVAE which learns a discrete “codebook” for converting speech into tokens.
2. An autoregressive decoder which predicts a probability distribution for speech tokens from (1) given text tokens.
3. A DDPM which can convert speech tokens back into speech spectrograms.
4. A contrastive model similar to CLIP which is used to rank outputs of the autoregressive decoder.

The architectures and training process for all of these networks largely follow the procedures found in their respective literature. Details can be found in appendix III.

Conditioning Input

A unique design choice made with TorToiSe is an additional input which is provided to both the autoregressive generator and the DDPM, which I call the “conditioning” input. The conditioning input starts as one or more audio clips of the same speaker as the target.

These clips are converted to MEL spectrograms and fed through an encoder consisting of a stack of self-attention layers. The autoregressive generator and the DDPM have their own conditioning encoders, both of which are trained alongside their respective networks.

The output of these layers is averaged to produce a single vector. The vectors from all of the encoded conditioning clips are then averaged again before being fed as an input into the autoregressive or conditioning networks.

The intuition behind the conditioning input is that it provides a prior that the models can use to infer vocal characteristics like tone and prosody such that the search space of possible speech outputs corresponding to a given textual input is greatly reduced.

Fine-tuning the DDPM on the Autoregressive Model Outputs

For the majority of the training procedure, the DDPM is trained to convert discrete speech codes into MEL spectrograms. After this training has converged, I fine-tune the DDPM on the autoregressive latent space which is pulled from the GPT model outputs instead of the speech codes. This is described in detail in appendix III.

This fine-tuning is one of the greatest contributors to model output quality of any of the tweaks I made to the various model training processes.

CLVP

As mentioned earlier, a good strategy for gathering expressive outputs from generative models is using a qualitative discriminator to re-rank several outputs, then choosing only the best. DALL-E uses CLIP for this.

This same type of approach used for CLIP can be easily applied to speech: after all, most TTS datasets are simply pairings of an audio clip and some text. By training a model on these pairs in a contrastive setting, the model becomes a good discriminator for speech.

For Tortoise, I train the Contrastive Language-Voice Pretrained Transformer, or CLVP. It has many of the same properties of CLIP, but notably serves as a scoring model for use in re-ranking TTS outputs from the AR model.

To make this work efficiently in inference, I trained CLVP to pair discretized speech tokens with text tokens. This way, CLVP can rerank multiple AR outputs without the expensive diffusion model being invoked.

Training

These models were trained on a small cluster of 8 NVIDIA RTX-3090s over the period of ~ 1 year.

Specifics on how these models are trained can be found in Appendix II.

Inference Process

Once the four models of the framework are fully trained, the inference procedure is as follows:

1. Feed the conditioning inputs and the text into the autoregressive model and decode a large number of output candidates.
2. Use CLVP to produce correlation scores between each speeech candidate and text.
3. Choose the top k speech candidates, for each candidate:
    a. Decode to a MEL spectrogram using the DDPM.
    b. Convert to a waveform using a conventional vocoder.

When decoding the autoregressive model, nucleus sampling is used with P=.8, repetition penalty=2 and softmax temperature=.8.

Sampling from DDPMs is a highly studied and rapidly changing field. At the time Tortoise was designed, I found the sampling configuration with the best balance between quality and inference speed to be as follows:

1. Algorithm: DDIM
2. Schedule: Linear
3. Sampling steps: 64
4. Conditioning-Free Guidance constant: 2

The Dataset

Since my goal was to train what is essentially a large language model, I needed a lot of data. I started with the LibriTTS and HiFiTTS datasets, which combined contain ~896 hours of transcribed speech. I built an additional, “extended” dataset of 49,000 hours of speech audio from audiobooks and podcasts scraped from the internet. Details on how this dataset was built are in appendix I. The official LibriTTS test split was used for validation purposes.

Experiments

Text to speech systems are challenging to experimentally compare because many state of the art systems are closed source with few samples to compare against. To this end, I built my own evaluation suite which uses CLVP to produce a distance metric between real samples and generated samples, similar to the FID score used by images. I also use an open source wav2vec model to characterize the “intelligibility” of a speech segment. I have open sourced this work here.

Past this, comparisons between the samples generated from Tortoise and those generated by other papers can be found here.

Conclusion

Tortoise is the latest in a line of recent state-of-the-art breakthroughs that use general model architectures. Almost no part of Tortoise was designed specifically for audio processing, yet it outperforms all known TTS models in quality. It does this by:

1. Embracing generalist architectures like stacks of transformer layers.
2. Leveraging a large, high-quality dataset.
3. Training at large-ish scale and high batch size.

My main take-away from this project is how incredibly strong the results are from adhering to the above 3 points. It seems likely to me that any digitized modality is subject to generative modeling using this framework.

References

Appendices

Appendix I - Extended Dataset Collection

I independently built an extended TTS dataset composed of audiobooks and podcasts scraped from the web. This data was split on 500ms silences, and any audio clip between 5-20 seconds was kept. I then fed the resulting clips through a pipeline of classifiers that I trained which remove any audio with background noise, music, poor quality (such as phone calls), multiple voices speaking at once and reverb. Due to disk space limitations, I was forced to limit the amount of scraping. The end result was 49,000 hours of cleaned audio clips.

I transcribed this dataset using a wav2vec2-large model. I personally fine-tuned this model to predict punctuation, as quotation marks, commas and exclamation marks are important for the purposes of generating speech but are not generally included in the training of speech recognition models. Fine-tuning was performed on LibriTTS and HiFiTTS and the pretrained model weights and transcription scripts can be found here.

Appendix II - Training and Architecture Details

VQVAE

The VQVAE used with Tortoise is most similar to that of the original VQVAE by van der Oord et al. It operates on MEL spectrograms. It consists of a small residual convolutional network that

compresses the spectrogram an additional 4x and produces a codebook consisting of 8192 tokens.

When training the VQVAE, I found that larger batch sizes decrease reconstruction losses, and thus used an very large batch size for my infrastructure. Input samples were constricted to 40960 PCM readings, or ~2 seconds of audio. The primary bottleneck for training the VQVAE was the dataloader.

1. Model shape: 1D Conv resnet, encoder + decoder. Top dim: 512. Bottom dim: 1024. Codebook dim: 256.
2. # Parameters: 51M
3. Quantizer token count: 8192
4. Quantization algorithm: Clustering + LUT, no restart.
5. Batch size: 8192
6. Total training: 360M samples
7. Losses: MSE reconstruction loss, commitment loss
8. Optimizer Hyperparameters:
    a. AdamW, Pytorch implementation with modifications to not apply WD to non-weight parameters.
    b. LR: 3e-4
    c. B1, B2: .9 .9999
    d. Weight decay: .01
    e. EMA weights replaces LR decay with rate: .999

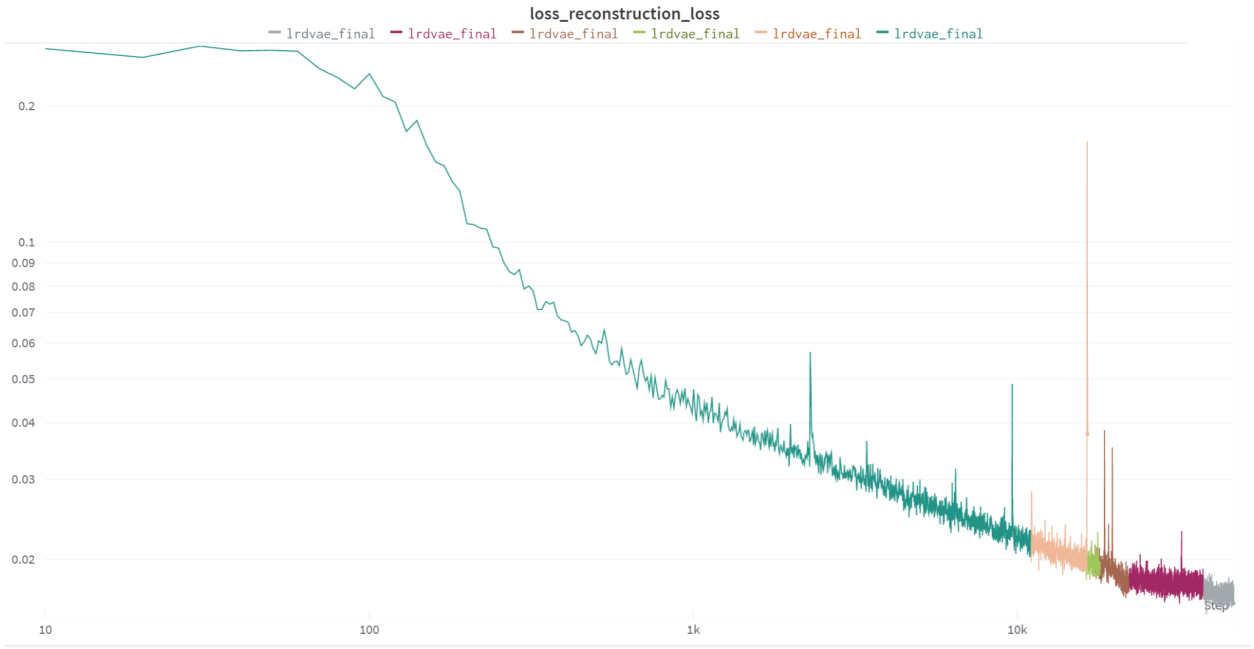


Figure A1: Training curves for VQVAE. Y-axis is MSE loss in log-log scale. X-axis is number of training steps.

Autoregressive Decoder

The AR decoder uses a bog-standard GPT-2 architecture and generally follows the training instructions from the DALLE-1 paper. Unlike DALL-E, only dense self-attention is used. The prompt is assembled as follows:
<SC><BT><T><T><T>..<T><ET><BM><M><M><M>...<EM>
SC=Speech conditioning encoding
BT=Begin text token
T=Text tokens
ET=End text token
BM=Begin MEL token
M=MEL tokens
EM=End MEL token

Speech conditioning encodings are learned by a separate encoder that takes in the MEL spectrogram of a related clip (another clip of the same person speaking) and produces a single vector embedding that is placed at the front of the attention context. Two encodings were produced for each training sample, which are averaged together. The maximum input length to the conditioning encoder is 132,300 samples, or 6 seconds of audio.

Learned positional embeddings are used. The MEL tokens and the text tokens get their own positional parameters. Text inputs are unpadded, MEL tokens are right padded to conform the sequence length of each batch. The maximum sequence length is 402 text tokens + 604 MEL tokens. For efficiency reasons, in the first half of training, the model only saw <6 second audio clips. After this, audio clips up to the full length (~27 seconds) were seen.

Further training details follow.

1. Model shape: Transformer stack with causal masking, 30 layers, d=1024, 16 heads.
2. # Parameters: 421M
3. Text tokenization: Custom BPE, 256 tokens wide.
4. Batch size: 1024
5. Total training: 119M samples
6. Losses:
    a. Text, next token prediction, weight .01.
    b. MEL token, next token prediction, weight 1.
7. Optimizer Hyperparameters:
    a. AdamW, Pytorch implementation with modifications to not apply WD to non-weight parameters and enable LR warmup.
    b. LR: 1e-4
    c. B1, B2: .9 .96
    d. Weight decay: .01
    e. 500 step warmup
    f. EMA weights replaces LR decay with rate: .999

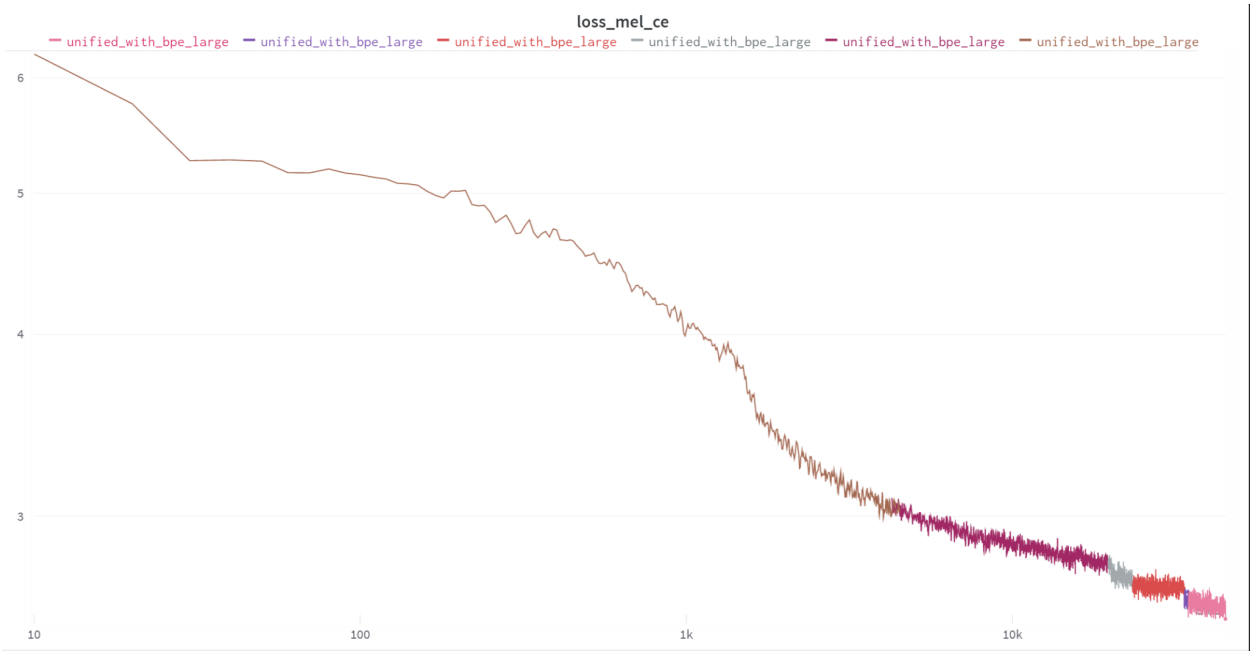


Figure A2: Early training curves in log-log scale. Y-axis is cross entropy loss for MEL tokens. X-axis is number of training steps. Does not include a long tail of training and fine-tuning due to online changes that were made adding non-reproducible noise to curves.

After training the autoregressive decoder to convergence, I fine-tuned it on the clean audio datasets from LibriTTS and HIFITTS.

CLVP

The original DALLE worked by decoding a large number of images for a given text prompt, which were then fed through CLIP. The image that CLIP deemed closest to the input text was used as the final output.

I continue following this lead for TorToiSe, for reasons that will become evident in the results section. I built a simple model that is very similar to CLIP, which I call a “Contrastic Language-Voice Pretrained” model, or CLVP. Like CLIP, this model produces distance metrics for text/speech pairs.

CLVP uses an architecture similar to the CLIP text encoder, except it uses two of them: one for text tokens and the other for MEL tokens. Tokens from both encoders were dropped out at a rate of 15%. Fixed positional embeddings were used. Maximum text input length was 350 tokens (in practice never actually seen). Maximum MEL token input length was 293, or ~13 seconds of audio.

Training details follow:

1. Model shape: Dual transformer stacks, full self attention. depth=20. d=768. num_heads=12.

2. # Parameters: N/A
3. Text tokenization: Custom BPE, 256-token wide
4. Batch size: 1024
5. Total training: 80M samples.
6. Losses: Contrastive
7. Optimizer Hyperparameters:
    a. AdamW, Pytorch implementation with modifications to not apply WD to non-weight parameters and enable LR warmup.
    b. LR: 3e-4
    c. B1, B2: .9 .96
    d. Weight decay: .001
    e. 500 step warmup
    f. EMA weights replaces LR decay with rate: .999

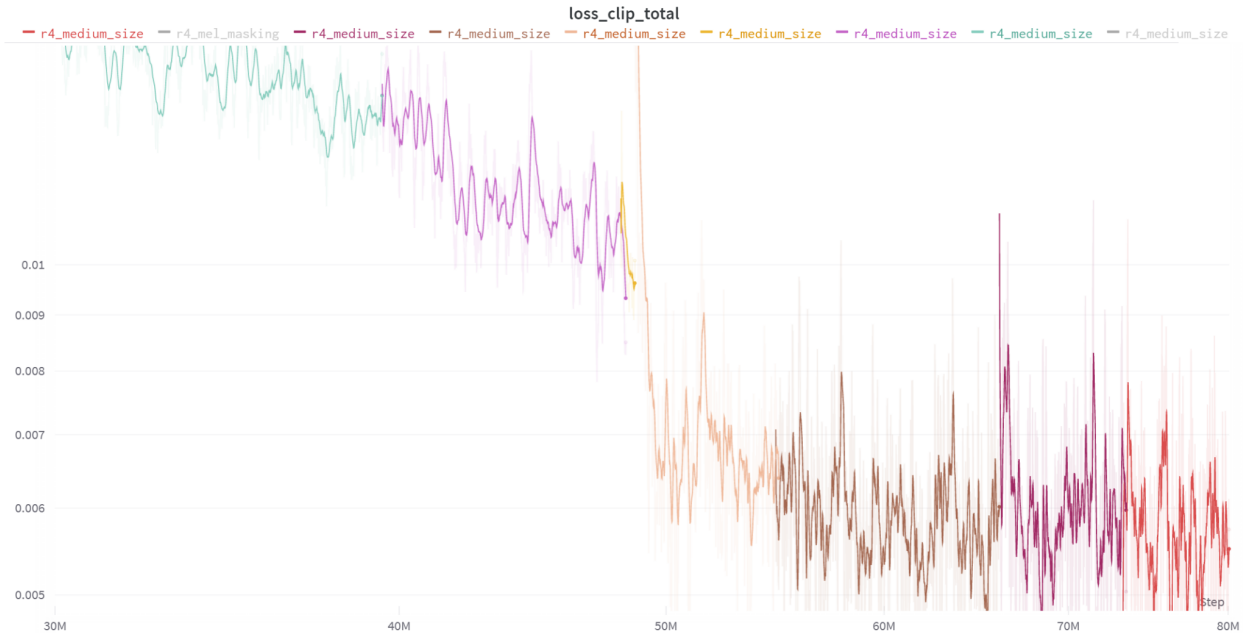


Figure A3: Late training curves for CLVP in log-log scale. Y-axis is cross entropy loss. X-axis is number of samples. Early training curves were lost.

Diffusion Decoder

The diffusion model uses a bespoke architecture that combines residual convolutions with dense self-attention. It most closely resembles the traditional U-Net model used for DDPMs but without any upsampling or downsampling.

The diffusion model receives 3 sources of conditioning:

1. The timestep signal, which modulates the scale and shift of the group norms used by the network.
2. A speech conditioning signal, which also modulates the scale and shift of the group norms.
3. The final activations of the autoregressive model.

In training the diffusion model, I iterated through several different architectures and conditioning types before settling on this one. This includes:

- Architecture: A “traditional” U-net with attention was tried. The full attention network performed significantly better in frechet distance evaluations.
- Operating on PCM data rather than MELs. This necessitated very small context windows and still took an inordinant amount of time to train. The results of decoding a MEL and using a vocoder resulted in substantially better quality. In order to force compatibility with existing diffusion noise schedules, I rescale input MELs to be on the interval [-1,1].
- Decoding MEL tokens versus AR activations. Training on AR activations is expensive because during each training step you must forward prop through the AR network. However, training on AR activations constituted the single greatest jump in output quality of any design decision made for the diffusion network. It is possible that doing tricks like putting the text on the attention context may ablate this advantage somewhat.

As with image diffusion models, exploiting classifier-free guidance is extremely important for high quality outputs. In the case of Tortoise, I perform guidance on both the speech conditioning signal and the activations of the AR model. During training, 15% of the time, both of these signals are dropped out and replaced with a learned embedding.

When training the diffusion decoder, input audio was clipped randomly to 220,500 samples, or 10 seconds of audio. Conditioning inputs were clipped to 102,400 samples, or ~5 seconds of audio.

While the rest of the Tortoise stack operates at an audio sampling rate of 22kHz, the diffusion decoder outputs MEL spectrograms which were computed from 24kHz audio. This discrepancy is solely to ensure compatibility with the pretrained Univnet vocoder which the model stack uses, and was not done for any performance reasons.

Further training details follow:

1. Model shape: Alternating full attention + conv resblocks. depth=10, d=1024, num_heads=16.
2. # Parameters:
3. Batch size: 512
4. Total Training: 65M samples
5. Losses: MSE (weight 1) + VLB (weight n)
6. Optimizer Hyperparameters:
    a. AdamW, Pytorch implementation with modifications to not apply WD to non-weight parameters and enable LR warmup.
    b. LR: 1e-5
    c. B1, B2: .9 .999
    d. Weight decay: .001
    e. 1000 step warmup
    f. EMA weights replaces LR decay with rate: .999

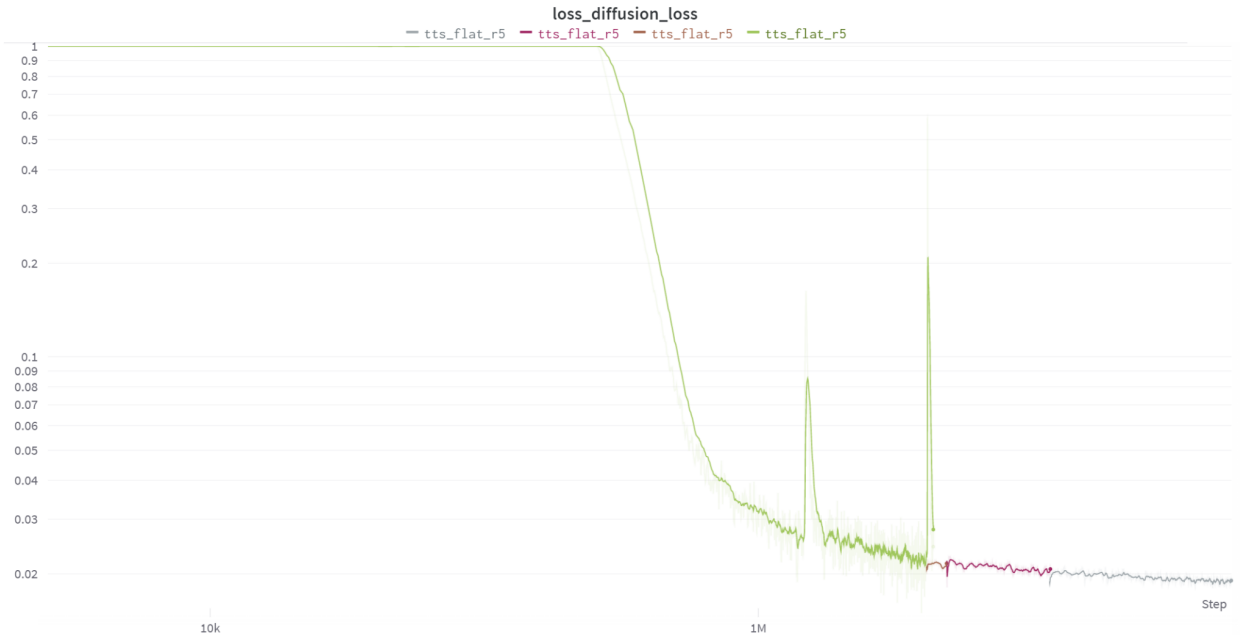


Figure A4: Diffusion model losses, log-log scale. Y-axis: MSE loss, X-axis: training samples.

Appendix IV - Future Work

Tortoise is the product of playing way over my paygrade, so to speak. As an independent researcher, I only had a small number of GPUs to perform my experiments with, and made many mistakes in the process. Following are recommendations for architectural tweaks to be made in future work building off of Tortoise:

1. Constrict VQVAE codebook dim. This has been experimentally shown to produce drastic performance improvements.
2. Relative positional encodings. The AR model uses fixed positional encodings, which limits the total amount of speech it can produce. Using relative encodings would allow arbitrary length sequences.
3. Train CLVP on larger batch sizes. Contrastive models benefit from extremely large batch sizes.
4. Train CLVP on longer audio sequences. CLVP only ever saw 13 second clips, which is likely why re-ranking on longer samples suffers.
5. Train the entire model stack at 24kHz or re-train Univnet at 22kHz sampling rates.